

Impact of Pivot Choice on QuickSort Performance

Introduction

The choice of pivot in **Quicksort** greatly affects its runtime performance. With a good pivot, the algorithm performs in $\mathcal{O}(n \log n)$ time. But with poor pivot choices, it can degrade to $\mathcal{O}(n^2)$.

We analyze the impact of three common pivot selection strategies.

1. Choosing the *First Element* as Pivot

Strategy

Always select the first element of the subarray as the pivot.

Potential Problem

- If the array is sorted (ascending or descending), each partition is unbalanced.
- One side has $n - 1$ elements, the other has 0.
- Leads to recursion depth of n , causing worst-case performance.

Time Complexity

- **Best case:** $\mathcal{O}(n \log n)$
- **Average case:** $\mathcal{O}(n \log n)$
- **Worst case:** $\mathcal{O}(n^2)$

Summary

This strategy is deterministic and vulnerable to sorted input. Generally not recommended.

2. Choosing the *Last Element* as Pivot

Strategy

Select the last element in the current subarray.

Notes

- Symmetric to the first-element strategy.
- Suffers the same unbalanced partitions on sorted input.

Time Complexity

- **Best/Average case:** $\mathcal{O}(n \log n)$
- **Worst case:** $\mathcal{O}(n^2)$

Summary

Also deterministic and poor for ordered data. Avoid in practice unless preprocessing is done.

3. Choosing a *Random Element* as Pivot

Strategy

Randomly select any element from the subarray as the pivot.

Advantages

- Less likely to consistently make poor splits.
- Performance becomes independent of input order.
- Expected to split the array roughly in half.

Time Complexity

- **Best case:** $\mathcal{O}(n \log n)$
- **Average case:** $\mathcal{O}(n \log n)$
- **Worst case:** $\mathcal{O}(n^2)$ (rare)

Summary

Most robust method. Avoids worst-case behavior with high probability. Recommended in most implementations.

Final Comparison

Pivot Choice	Worst Case	Average Case	Robustness
First element	$\mathcal{O}(n^2)$	$\mathcal{O}(n \log n)$	X Poor (sorted input fails)
Last element	$\mathcal{O}(n^2)$	$\mathcal{O}(n \log n)$	X Poor (sorted input fails)
Random element	$\mathcal{O}(n^2)^*$	$\mathcal{O}(n \log n)$	✓ Excellent (rare worst case)

*Worst-case time still exists in theory, but is extremely rare unless unlucky random choices are made repeatedly.